



VISÃO COMPUTACIONAL

AULA 6



Prof. Leonardo Gomes



CONVERSA INICIAL

Nesta aula discutiremos uma variedade de possíveis aplicações para visão computacional. Vamos combinar técnicas vistas anteriormente para solucionar alguns problemas comuns para sistemas de visão computacional. Ao final, esperamos atingir os seguintes objetivos que serão avaliados ao longo da disciplina da forma indicada no Quadro 1 a seguir.

Quadro 1 – Objetivos e formas de avaliação

Objetivos	Avaliação
1 – Entender e aplicar captura de texto por OCR por meio do Tesseract.	Questionário e questões dissertativas
2 – Saber aplicar a detecção de objetos utilizando pontos característicos como SIFT, SURF e ORB	Questionário e questões dissertativas
3 – Experimentar a aplicação de técnicas para rastreamento de objetos em vídeo	Questionário e questões dissertativas

TEMA 1 – DETECÇÃO E DESCRIÇÃO DE PONTOS CARACTERÍSTICOS

Alguns dos mais importantes algoritmos para extração e detecção de características e descritores estão implementados no *opencv* e podem ser convenientemente acessados por meio de poucas linhas de código. Por exemplo, o Haar Features visto anteriormente é um desses algoritmos, e embora Haar seja rápido e possa ser treinado para uma série de objetos distintos, seu uso geralmente é mais indicado para classes de objetos com certo padrão e que estejam em determinada orientação específica; uma base de treinamento para faces verticais não reconhecerá um rosto que apareça na horizontal.

Outra limitação é que se faz necessário saber de antemão o objeto desejado para que se realize o treinamento. Se desejamos uma solução para seguir o deslocamento de objetos ou pontos característicos em uma sequência de imagens e não sabemos previamente qual será o objeto, Haar também não é recomendado. Então, se queremos buscar na imagem um objeto específico e independente de sua escala e rotação na imagem – como a capa de um livro em um vídeo de celular que captura a cena por diversos ângulos –, temos outros



algoritmos mais apropriados, como *Scale-Invariant Feature Transform* (SIFT), *Speeded Up Robust Features* (SURF) e *Oriented FAST and Rotated BRIEF* (ORB).

Embora o exemplo dado em um primeiro momento pareça ser bastante específico, na verdade esse cenário descrito envolve a solução de um problema que aparece em muitas aplicações distintas, como realidade aumentada, mapeamento robótico tridimensional ou geração de imagens panorâmicas.

- **SIFT** – proposto pelo pesquisador Lowe em 1999, possui quatro passos básicos: primeiro, cria um espaço de escala no qual aplica uma sucessão de filtros gaussianos; na sequência pontos-chave candidatos são localizados, e os de menor contraste são eliminados; por terceiro, a orientação dos gradientes dos pontos são definidos; e, por fim, o descritor para cada ponto-chave é gerado.
- **SURF** – proposto por Bay et al. em 2006, utiliza estratégias semelhantes ao SIFT, porém se propõe a ser mais rápido, utilizando descritores com menos características, combinando estratégias de matrizes hessianas que aumentam a robustez do método e detectores de cantos para selecionar os candidatos para pontos característicos.
- **ORB** – proposto por Rublee et al. em 2011, combinou estratégias de um algoritmo de detecção de pontos-chave chamado FAST com o descritor de outro algoritmo chamado BRIEF. Em seu artigo, os autores apresentaram resultados que indicam desempenho superior ao SIFT e SURF em termos de uso de processamento.

No *opencv*, os três algoritmos podem ser carregados na memória em um objeto gerenciador com um comando simples conforme mostrado a seguir. O objeto retornado é responsável por armazenar os parâmetros da execução e os métodos relacionados.

```
sift = cv2.SIFT_create()
surf = cv2.SURF_create()
orb = cv2.ORB_create()
```



Importante!

Caso sua versão do *opencv* não esteja reconhecendo os comandos relacionados, recomendamos desinstalá-la e instalar a versão **contrib** com pacotes extras por meio dos comandos:

- pip uninstall opencv-python
- pip install opencv-contrib-python

Na sequência, os comandos descritos na sequência são utilizados para detectar os pontos-chave, aqueles pontos na imagem cuja vizinhança possua um padrão de intensidade luminosa específico. O comando *detectAndCompute* recebe a imagem como parâmetro e a região de interesse da imagem na forma de um retângulo; quando *None* é passado, ele considera a imagem completa. O método retorna uma lista com os pontos-chave, com todas as informações relevantes e uma lista equivalente com os respectivos descritores.

```
kpSift,descriptor= sift.detectAndCompute(img, None)
kpSurf,descriptor= surf.detectAndCompute(img, None)
kpOrb,descriptor= orb.detectAndCompute(img, None)
```

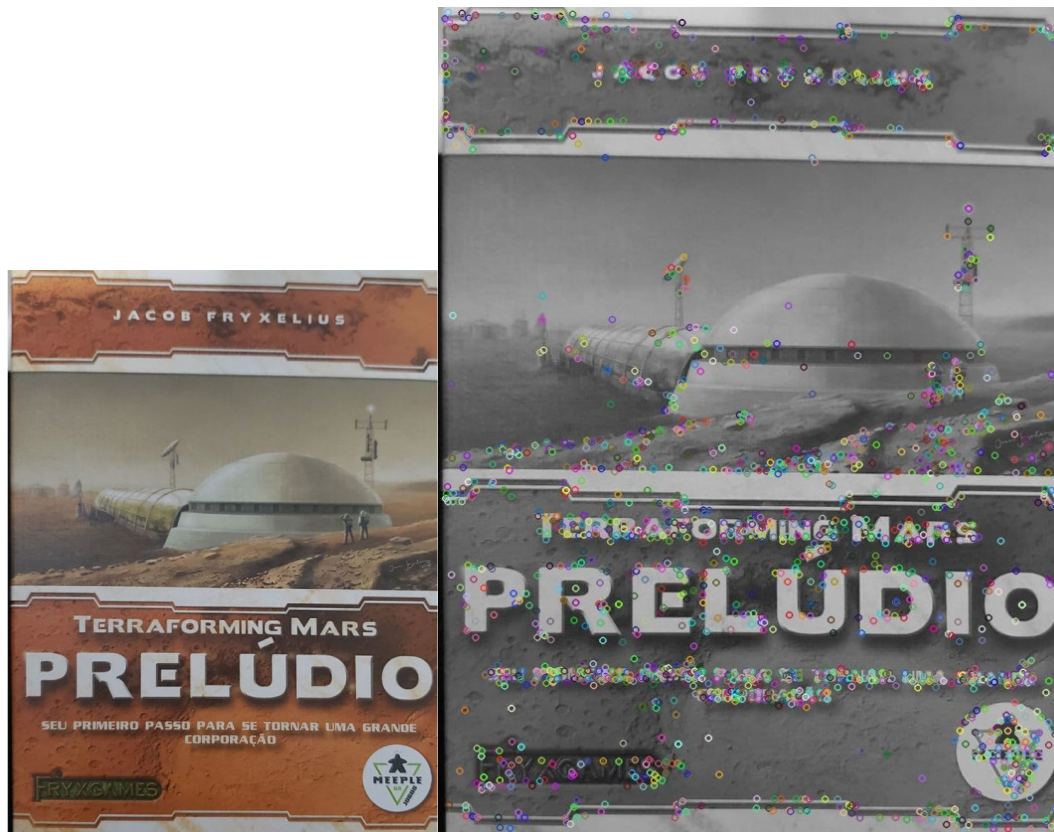
São esses os pontos-chave que serão posteriormente comparados aos de outras imagens, permitindo realizar um rastreamento deles.

Por meio dos comandos a seguir, podemos visualizar esses pontos-chave.

```
img = cv2.drawKeypoints(img, kp, None)
cv2.imshow("Image", img)
```



Figura 1 – Imagem original da caixa de um jogo de tabuleiro; junto dela os pontos-chave detectados pelo algoritmo SIFT marcados em diversas cores



Crédito: Jacob Fryxlius/Amazon.

TEMA 2 – COMPARAÇÃO DE PONTOS CARACTERÍSTICOS

Agora que sabemos como encontrar os pontos-chave, nos resta compará-los com os pontos de outra imagem e assim detectar onde está o nosso objeto. Um dos métodos possíveis de fazer isso é o que chamamos de força bruta, que consiste em comparar todos os pontos contra todos os pontos, e aqueles que forem semelhantes são classificados como combinações. No *opencv*, a força bruta já é implementada por meio do comando *BFMatcher*, o primeiro parâmetro indica o método pelo qual será medida a diferença entre os pontos-chave, e *cv2.NORM_HAMMING* é o parâmetro que sugere a aplicação de um método que considera a diferença entre duas entidades como a quantidade de trocas que é necessário fazer para que se tornem iguais.

Na sequência, temos o parâmetro opcional *crossCheck*; por padrão, ele é falso, mas quando marcado verdadeiro as combinações serão filtradas, e apenas aquelas que mutuamente se reconheçam como as mais semelhantes serão utilizadas. Imagine em um ponto X na primeira imagem e que para ele o ponto-



chave mais semelhante na segunda imagem seja um ponto Y; porém, se esse ponto Y entender que o ponto mais semelhante a ele na primeira imagem é um outro ponto Z, então essa combinação (X,Y) será descartada; apenas quando ambos os pontos se entendem os mais semelhantes, a combinação será mantida. O método produz um objeto responsável por aplicar a estratégia de casamento de pares de pontos-chave.

Na sequência, temos o método *match*, que recebe duas listas de descritores e faz os pareamentos seguindo a configuração do seu objeto e retorna uma lista com os pares de descritores combinados por semelhança.

```
fb=cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
combinacoes = fb.match(des1, des2)
```

Os pares de combinações podem ser muitos, dependendo da natureza das imagens, e com frequência os pareamentos não são corretos. Então, o que podemos fazer é organizar as combinações por ordem de semelhança: os pontos-chave mais semelhantes ficam nas primeiras posições, e utilizamos um conjunto limitado desses pontos mais semelhantes. No código a seguir isso é feito em *opencv*. O método *lambda* retorna a *distance* (distância), que é a diferença entre os dois pontos, e com o comando *drawMatches* as combinações são ilustradas na imagem de resultado, porém no código a seguir apenas as dez mais semelhantes; o parâmetro *flags=2* remove os pontos-chave que não fazem parte das combinações.

```
combinacoes=sorted(combinacoes,
                    key=lambda x: x.distance)
resultado = cv2.drawMatches(img1, kp1, img2, kp2,
                             combinacoes[:10], None, flags=2)
```

A seguir, apresentamos o código completo e o respectivo resultado ilustrando a imagem de uma caixa de um jogo de tabuleiro tendo seus pontos-chave adequadamente detectados em meio a uma estante com diversos outros jogos, mesmo estando rotacionado e com significativa diferença de escala.



```
import cv2
img1 = cv2.imread("jogo.jpg", 0)
img2 = cv2.imread("estante1.jpg", 0)
orb = cv2.ORB_create()

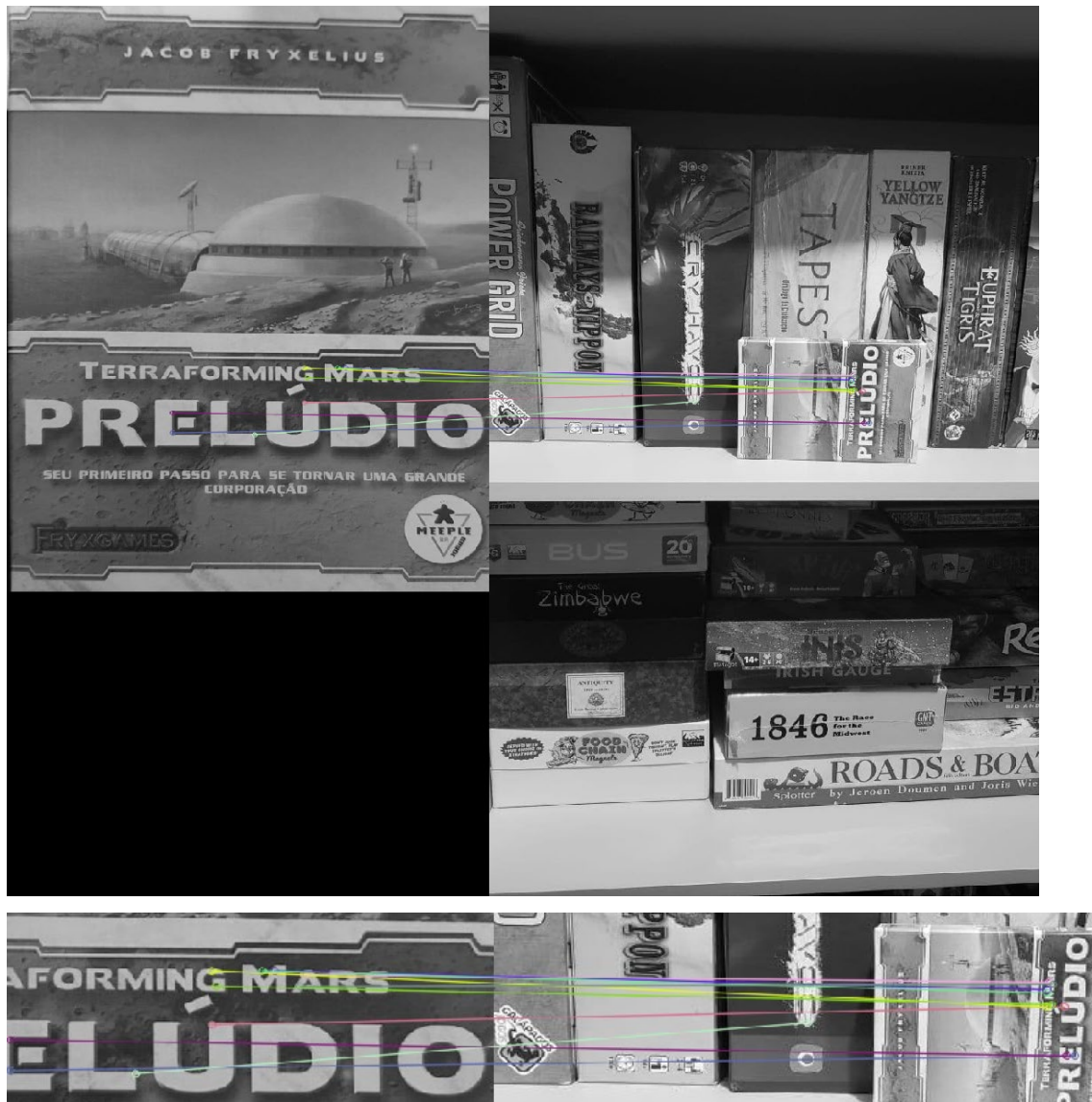
kp1, des1 = orb.detectAndCompute(img1, None)
kp2, des2 = orb.detectAndCompute(img2, None)

fb=cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
combinacoes = fb.match(des1, des2)
combinacoes=sorted(combinacoes,
                    key=lambda x: x.distance)
resultado = cv2.drawMatches(img1, kp1, img2, kp2,
                             combinacoes[:10], None, flags=2)
cv2.namedWindow('resultado', cv2.WINDOW_NORMAL)
cv2.imshow("resultado", resultado)
cv2.waitKey(0)
```

Na Figura 2, temos a imagem original da caixa de um jogo de tabuleiro (esquerda) e uma imagem de uma estante repleta de vários jogos (direita); os dez pares de pontos-chave mais semelhantes foram associados, e o objeto original adequadamente localizado, apesar de estar rotacionado; na parte de baixo, um *zoom* na região específica.



Figura 2 – Resultado do método de paramento



Crédito: Leonardo Gomes.

TEMA 3 – RECONHECIMENTO DE CARACTERES

Outra área de pesquisa importante dentro da visão computacional é o uso de *Optical Character Recognition* (OCR), que é o reconhecimento automático de caracteres em imagens. As aplicações são diversas, como criar um tradutor ou leitor automático pela câmera, digitalizar notas fiscais para realizar cálculos automatizados, reconhecimento de placas de automóveis para efetuar buscas em bancos de dados etc.

Nos últimos anos, em especial depois do crescimento de tecnologias de classificação com uso de *deep learning*, os algoritmos de OCR têm se tornado muito eficientes. Um dos *softwares* mais relevantes no mercado referente a OCR



é o Tesseract, com implementação em código aberto e financiamento pela Google. A solução também passou a adotar *deep learning* após a versão 4.0 do *software*.

Saiba mais

Os instaladores, assim como seu código, estão disponíveis para *download* no github (<<https://github.com/tesseract-ocr/tesseract>>. Acesso em: 3 fev. 2022).

No momento em que este material é produzido, o *software* se encontra na versão *alpha* 5.0, e a mais estável está na 4.1.1. No momento da instalação, é importante escolher também os idiomas que deseja utilizar, pois cada um possui uma base de treinamento diferenciada para a qual é otimizada; por padrão, apenas inglês é instalado. Para sua utilização em conjunto ao código *python*, é necessário primeiro que seja instalado o Tesseract, e, na sequência, uma biblioteca, *pytesseract* que se conecta ao *software* já instalado. O comando de instalação para a biblioteca apenas é:

```
pip install pytesseract
```

Feita essa instalação, é possível conectar as imagens com o *software* do Tesseract já instalado; apenas a biblioteca não basta.

O código a seguir é um exemplo *python* que utiliza Tesseract para aplicação de OCR na foto de um livro. A biblioteca *pytesseract* faz a conexão com o *software* Tesseract enquanto a biblioteca PIL carrega a imagem no formato compatível com o *pytesseract*. Os formatos de imagem *opencv* não são compatíveis aqui. Na linha de comando: `pytesseract.pytesseract.tesseract_cmd` passamos por parâmetro o endereço de onde o Tesseract foi instalado. O comando `pytesseract.image_to_string(img, lang="por")` faz a transformação da imagem em string. Recebe por parâmetro a imagem e opcionalmente o idioma; caso nenhum idioma seja passado por parâmetro inglês, será assumido como padrão.

```
import pytesseract

from PIL import Image

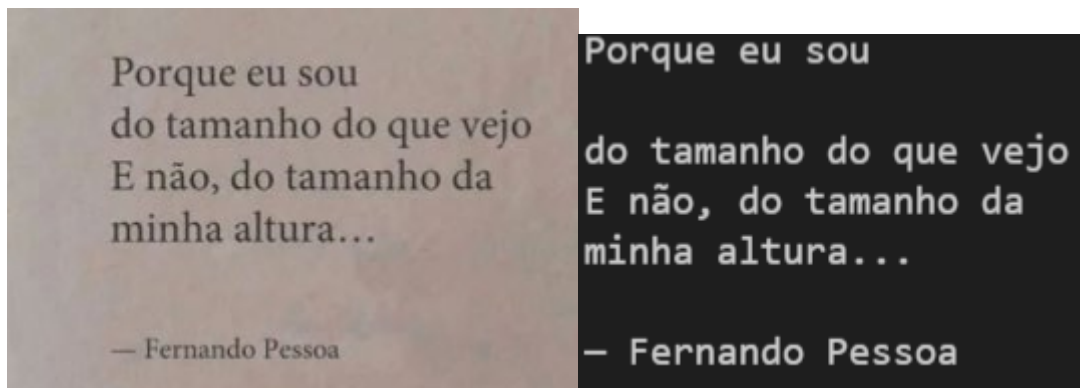
pytesseract.pytesseract.tesseract_cmd=
r'D:\Tesseract-OCR\tesseract.exe'

img = Image.open("fpessoa.png")
```



```
texto= pytesseract.image_to_string(img,lang="por")  
print(texto)
```

Figura 3 – Foto de um livro contendo o trecho do poema "O Guardador de rebanhos", de Fernando Pessoa (esquerda) e o mesmo trecho digitalizado com todos os caracteres sendo adequadamente reconhecidos (direita)



Crédito: Leonardo Gomes.

TEMA 4 – RASTREIO DE OBJETO

Outro problema importante que as técnicas de visão computacional ajudam a resolver é o rastreamento de objetos em vídeo. Seja para contagem de carros que passam em uma avenida, seja para identificar pessoas em uma câmera de segurança, são diversas as aplicações.

No *opencv*, diferentemente do que vimos com outras situações, não existe uma única função que seja aplicada e resolva o problema do rastreamento de objetos. São muitas as variáveis e situações particulares para cada cenário. Ao longo das próximas páginas vamos rastrear carros que passam em uma avenida e contabilizar o total deles utilizando o vídeo de uma câmera fixa.

O código que vamos analisar foi dividido em dois arquivos. O primeiro é **rastreador.py**, que possui uma classe para ajudar a contabilizar quais objetos estão na tela no momento e quais seus índices; para facilitar o entendimento, deixaremos essa análise por último. O segundo arquivo é o **main.py**, que é o nosso arquivo principal e que fará todo o restante do trabalho (carregar vídeo, separar região de interesse, fazer uso da classe Rastreador etc.).

```
rastreador = Rastreador()  
cap = cv2.VideoCapture("rua.mp4")
```



O código começa com a importação e instanciação do objeto Rastreador e inicialização da captura do vídeo rua.mp4.

```
detector = cv2.createBackgroundSubtractorMOG2(  
    history=100, varThreshold=40)
```

Na sequência, capturamos o fundo da imagem. Como estamos trabalhando em um cenário de câmera fixa, a estratégia mais simples de detecção de objetos é pelo seu movimento; a subtração da imagem de fundo pela imagem atualizada oferece uma forma rápida e eficiente de encontrar os veículos na cena. O *opencv* oferece uma ferramenta fácil para esse tipo de detecção por meio do método *createBackgroundSubtractorMOG2*. O detector vai gerar uma imagem com *pixels* brancos para representar movimento, e pretos para regiões estáticas; os parâmetros *history* marcam quantos *frames* serão armazenados para o histórico do fundo da imagem, e *varThreshold*, o quão diferente o *pixel* pode ser para ainda ser considerado fundo.

```
while True:  
    ret, frame = cap.read()  
    altura, largura, _ = frame.shape
```

No *loop* principal do código fazemos a captura de uma imagem nova, assim como suas características de largura e altura em *pixels*.

```
roi = frame[812: 918, 702: 870]
```

Como a câmera captura uma área muito grande, é interessante limitar a detecção para uma região de interesse. Visto que nossa câmera é fixa, esse valor foi definido especificamente para esse vídeo que marca uma área retangular em um trecho da avenida. A variável aqui foi chamada de ROI por ser um acrônimo bastante comum que vem da expressão em inglês *Region of interest* (região de interesse). Os parâmetros são as coordenadas em *pixels* do ponto superior esquerdo da imagem e ponto inferior direito.

```
mascara = detector.apply(roi)  
_, mascara = cv2.threshold(mascara, 254, 255,  
    cv2.THRESH_BINARY)
```

A região de interesse então é passada ao detector, que retorna a imagem em tons de cinza. O fundo é representado em tons escuros, ao passo que o que



é objeto que entrou na cena utiliza tons claros. Na sequência, para analisarmos melhor, transformamos a imagem em preto e branco propriamente, qualquer *pixel* com cor abaixo de 254 é considerado preto, ou seja, fundo.

```
contornos, _ = cv2.findContours(mascara,
cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
detectados = []
for cnt in contornos:
    area = cv2.contourArea(cnt)
    if area > 400:
        x, y, w, h = cv2.boundingRect(cnt)
        detectados.append([x, y, w, h])
```

Na sequência, cada objeto é segmentado pelas bordas, e apenas aqueles com tamanho superior a 400 *pixels* de área são considerados como carros e anexados na lista de objetos detectados.

```
ids_caixas = rastreador.atualizar(detectados)
for id_caixa in ids_caixas:
    x, y, w, h, id = id_caixa
    cv2.putText(roi, str(id), (x, y -
15), cv2.FONT_HERSHEY_PLAIN, 2, (255, 0, 0), 2)
    cv2.rectangle(roi, (x, y), (x + w, y + h), (0,
255, 0), 3)
```

O objeto rastreador é atualizado com a lista de objetos encontrados, e então uma caixa/retângulo é desenhada em volta de cada um e um texto contém o índice do objeto.

```
cv2.imshow("roi", roi)
cv2.imshow("Frame", frame)
cv2.imshow("Mask", mascara)
key = cv2.waitKey(30)
if key == 27:
    break
```



```
cap.release()
cv2.destroyAllWindows()
```

Por fim, o programa é encerrado apresentando as janelas e aguardando a tecla ESC, código 27, ser pressionada. O código completo está apresentado a seguir.

```
import cv2
from rastreador import *
# Criar objeto detector
rastreador = Rastreador()
cap = cv2.VideoCapture("rua.mp4")

# Deteccao de objetos em camera estatica
detector = cv2.createBackgroundSubtractorMOG2(
    history=100, varThreshold=40)

while True:
    ret, frame = cap.read()
    altura, largura, _ = frame.shape

    # Regiao de interesse
    roi = frame[812: 918, 702: 870]
    # 1. Deteccao de objeto
    mascara = detector.apply(roi)
    _, mascara = cv2.threshold(mascara, 254, 255, cv2.THRESH_BINARY)

    contornos, _ = cv2.findContours(mascara, cv2.RETR_TREE,
cv2.CHAIN_APPROX_SIMPLE)
    detectados = []
    for cnt in contornos:
        #area e remocao de elementos pequenos
        area = cv2.contourArea(cnt)
        if area > 400:
            x, y, w, h = cv2.boundingRect(cnt)
            detectados.append([x, y, w, h])

    # Rastreio de objeto
    ids_caixas = rastreador.atualizar(detectados)
    for id_caixa in ids_caixas:
        x, y, w, h, id = id_caixa
        cv2.putText(roi, str(id), (x, y - 15), cv2.FONT_HERSHEY_PLAIN, 2,
(255, 0, 0), 2)
```



```
cv2.rectangle(roi, (x, y), (x + w, y + h), (0, 255, 0), 3)

cv2.imshow("roi", roi)
cv2.imshow("Frame", frame)
cv2.imshow("Mask", mascara)

key = cv2.waitKey(30)
if key == 27:
    break

cap.release()
cv2.destroyAllWindows()
```

TEMA 5 – CLASSE PARA RASTREIO DE OBJETO

Vamos agora analisar a classe Rastreador, que foi utilizada no código principal.

```
import math
class Rastreador:
    def __init__(self):
        self.pontos_centrais = {}
        self.contador_id = 0
```

Ela é criada, e dois atributos são estabelecidos: uma lista com os pontos centrais dos objetos e uma lista com o contador que objetos, que também servirá para identificar cada objeto individualmente com um número próprio.

```
def atualizar(self, retangulos):

    bbs_ids = []

    for rect in retangulos:

        x, y, w, h = rect

        cx = (x + x + w) // 2

        cy = (y + y + h) // 2
```



O método principal da classe é o atualizar, responsável por receber os retângulos com os objetos capturados na cena e registrá-los. A lista, chamada `bb_ids[]`, vai guardar a identificação de cada objeto e as coordenadas dos retângulos que envolvem esses objetos. Esse retângulo é muito conhecido na literatura pelo nome de *bounding box*, expressão no inglês que se traduz como caixa envolvente. No código que acabamos de apresentar a posição central de cada objeto no eixo X e Y é calculada.

```
mesmo_objeto = False

for id, pt in self.pontos_centrais.items():

    dist = math.hypot(cx - pt[0], cy - pt[1])

    if dist < 25:

        self.pontos_centrais[id] = (cx, cy)

        print(self.pontos_centrais)

        bbs_ids.append([x, y, w, h, id])

        mesmo_objeto = True

        break
```

Na sequência, ainda dentro do *loop*, comparamos o ponto central de cada objeto detectado com os de outros objetos da última atualização, e caso esteja a uma distância menor do que 25 *pixels*, o objeto será considerado o mesmo da última atualização. Essa estratégia pode falhar se for a detecção de um objeto que se move muito rápido na tela ou se tivermos objetos se interpolando um sobre o outro. Em um cenário assim, pode ser interessante levar outras características em consideração além da posição, como a forma e a aparência do objeto, por exemplo.

```
if mesmo_objeto is False:

    self.pontos_centrais[self.contador_id] = (cx, cy)

    bbs_ids.append([x, y, w, h, self.contador_id])
```




```
self.contador_id += 1
```

E ainda no *loop*, caso o objeto seja novo, então um ID é criado para ele e adicionado na lista de objetos registrados, contendo as medidas da *bounding box* e do ID.

```
novos_pontos_centrais = {}  
for bb_id in bbs_ids:  
    _, _, _, _, identificador = bb_id  
    centro = self.pontos_centrais[identificador]  
    novos_pontos_centrais[identificador] = centro  
self.pontos_centrais = novos_pontos_centrais.copy()  
return bbs_ids
```

Por fim, os dados dos pontos centrais são atualizados, e os *bounding boxes*, retornados pelo método atualizar para que possam ser desenhados na tela junto dos respectivos índices. A seguir, confira o código completo do rastreador.py

```
import math  
class Rastreador:  
    def __init__(self):  
        # Armazena a posicao central dos objetos  
        self.pontos_centrais = {}  
        # Mantem o contador dos identificadores  
        # Cada vez que um novo objeto for detectado incrementa o id  
        self.contador_id = 0  
  
    def atualizar(self, retangulos):  
        # bounding box e ids  
        bbs_ids = []  
  
        # Centro dos objetos  
        for rect in retangulos:  
            x, y, w, h = rect  
            cx = (x + x + w) // 2
```

```

cy = (y + y + h) // 2

# Descubre se o objeto ja foi detectado antes
mesmo_objeto = False
for id, pt in self.pontos_centrais.items():
    dist = math.hypot(cx - pt[0], cy - pt[1])
    if dist < 25:
        self.pontos_centrais[id]=(cx, cy)
        print(self.pontos_centrais)
        bbs_ids.append([x, y, w, h, id])
        mesmo_objeto = True
        break

# Se novo objeto for descoberto definimos um ID
if mesmo_objeto is False:
    self.pontos_centrais[self.contador_id] = (cx, cy)
    bbs_ids.append([x, y, w, h, self.contador_id])
    self.contador_id += 1

# Reseta dicionario e remove IDs que nao sao mais utilizados
novos_pontos_centrais = {}
for bb_id in bbs_ids:
    _, _, _, _, identificador = bb_id
    centro = self.pontos_centrais[identificador]
    novos_pontos_centrais[identificador] = centro

# Atualiza dicionario, remove IDs nao utilizados
self.pontos_centrais = novos_pontos_centrais.copy()
return bbs_ids

```

Na Figura 4, temos um *frame* do método em execução. No centro da imagem duas pequenas janelas podem ser vistas, uma com imagem em preto e branco do objeto detectado na área de interesse, e outra, colorida, com o objeto contornado em verde e o índice contador em azul.



Figura 4 – *Frame* do vídeo da detecção de veículos



Crédito: Leonardo Gomes.

De forma geral, o código apresentado é bastante específico ao problema em questão. No entanto, a linha de raciocínio utilizada pode ser aplicada a outros problemas similares com pequenas variações.

FINALIZANDO

Ao longo desta aula, discutimos assuntos variados relativos à visão computacional. Conhecemos uma nova técnica para detectar objetos por meio de pontos característicos com os métodos SIFT, SURF e ORB. Vimos também um exemplo de rastreamento detectando e contando carros em vídeo com câmera estática e utilizando o movimento para identificar objetos. Além disso, verificamos a técnica de OCR sendo aplicada com a biblioteca Tesseract.

Mediante esses métodos e os demais estudados ao longo de nossa jornada de conhecimento, esperamos que você possa obter obtido uma noção, ainda que superficial, bastante prática das etapas e aplicações de visão computacional utilizando *opencv*.



REFERÊNCIAS

BARNES, D. J.; KÖLLING, M. **Programação orientada a objetos com Java**. 4. ed. São Paulo: Pearson Prentice Hall, 2009.

DEITEL, P.; DEITEL, H. **Java** – Como programar. 10. ed. São Paulo: Pearson, 2017.

LARMAN, C. **Utilizando UML e padrões**: uma introdução à análise e ao projeto orientados a objetos e ao desenvolvimento iterativo. 3. ed. Porto Alegre: Bookman, 2007.

MEDEIROS, E. S. **Desenvolvendo software com UML 2.0**: definitivo. São Paulo: Pearson Makron Books, 2004.

PAGE-JONES, M. **Fundamentos do desenho orientado a objetos com UML**. São Paulo: Makron Books, 2001.

PFLEEGER, S. L. **Engenharia de software**: teoria e prática. 2. ed. São Paulo: Prentice Hall, 2004.

SINTES, T. **Aprenda programação orientada a objetos em 21 dias**. 5. reimp. São Paulo: Pearson Education do Brasil, 2014.

SOMMERVILLE, I. **Engenharia de software**. 9. ed. São Paulo: Pearson, 2011.
